MSML/MSAI 605
Milestone 3 (Detailed)

What Milestone 3 is testing

Milestone 3 is where your face verification project becomes a deployable inference system. In Milestone 1, you built the reproducible plumbing: deterministic LFW ingestion, deterministic pair generation, and vectorized similarity scoring. In Milestone 2, you kept that backbone and added disciplined evaluation through threshold calibration, tracked runs, error analysis, data-centric iteration, validation checks, and tests.

In Milestone 3, you now move from the earlier baseline representation to an embedding-based face representation, while preserving the deterministic pair and evaluation backbone from Milestones 1 and 2. You will then package that verifier as a deployable artifact, expose a clean inference interface, and measure performance under concurrent usage.

This milestone is therefore not just about putting code into Docker. It is about turning your evaluated verifier into a usable inference system that can be run from a clean clone, exercised through a CLI, and characterized with a feasible local load test.

- Embedding-based inference, meaning the similarity score is now computed from face embeddings rather than only from a weak placeholder representation.
- Deployable packaging, meaning a grader can build and run the system through Docker without hand-fixing the environment.
- Clean inference interface, meaning the verifier exposes a readable CLI output for score, decision, confidence, and latency.
- Runtime characterization, meaning you can say how the system behaves under concurrent usage, not just whether it runs once.
- Reproducibility, meaning the README commands, tag, and artifact paths are clear enough for a grader to reproduce the milestone outputs from a fresh clone.

What you must submit in Canvas

- Your Git repository link.
- Your Milestone 3 Git tag (example: v0.3).
- If an important artifact path is not obvious from the README, make that location explicit in the README itself.

Definition (quick checklist)

- The repository still has a clean structure, a pinned environment, and a README with a brief project overview plus a clear How to run section.
- The verifier now uses an embedding-based representation for face images.
- The project reuses the deterministic pair and evaluation backbone from earlier milestones, unless you clearly document a justified change.

- The inference interface is packaged in Docker and runnable from a clean clone.
- A CLI is provided by default.
- CLI output includes a binary decision, similarity score, calibrated confidence, and latency for each pair.
- A concurrency or load-test script runs a feasible local workload and reports throughput and latency distribution.
- You created and pushed a Milestone 3 tag such as v0.3 that points to the reproducible commit.

1. How Milestone 3 fits into this project

A common mistake in Milestone 3 is to treat it as pure deployment work. To stay aligned with the course project, keep the full system view in mind.

- Milestone 1 built the deterministic system backbone.
- Milestone 2 built the thresholding, evaluation, and iteration loop.
- Milestone 3 upgrades the representation to embeddings and turns the verifier into deployable inference.
- Milestone 4 then audits, profiles, documents, and finalizes that near-complete system.

The table below summarizes what each milestone contributes to the project flow.

Milestone	**Main contribution to the project**
Milestone 1	Deterministic ingestion, saved pairs, reproducible structure, and fast vectorized scoring.
Milestone 2	Threshold calibration, tracked runs, error analysis, data-centric iteration, validation checks, and tests.
Milestone 3	Embedding-based inference, Docker packaging, CLI interface, and concurrency or load testing.
Milestone 4	System card, limitations and fairness discussion, hardware-aware profiling, and final reproducibility release.

2. Important Milestone 3 terms, in project context

Before you implement Milestone 3, make sure the terms below are clear in the context of this project.

Term	**What it means in this milestone**
Embedding	A fixed-length vector representation of a face image produced by your chosen embedding stage. Similarity should now be computed from these embeddings.
Inference interface	A user-facing way to run the verifier on input images or pairs. For this milestone, the default interface must be a CLI.
Pair-level inference	Running the verifier on one pair of images and returning score, decision, confidence, and latency.

Term	What it means in this milestone
Calibrated confidence	A documented confidence value derived from the score and operating threshold using a clearly explained rule.
Load test	A script that invokes inference repeatedly under multiple workers or processes and reports throughput plus latency behavior.
p95 latency	A tail-latency statistic showing how slow the slower requests are, not just the average.
Smoke test	A very small end-to-end check that confirms the main CLI path completes successfully.

3. Recommended repository additions for Milestone 3

You may keep the same overall repository layout from Milestones 1 and 2. Milestone 3 usually adds clearer separation for inference, embeddings, Docker, and load testing. The exact folder names can vary, but the responsibilities should be cleanly separated.

Area	What belongs there
README.md	Brief project overview, Milestone 3 scope, Docker and CLI instructions, load-test instructions, and artifact paths.
src/	Importable code for preprocessing, embeddings, scoring, thresholding, confidence computation, and inference logic.
scripts/	Runnable entrypoints such as the CLI wrapper, load-test script, and any helper used to prepare or evaluate pairs.
configs/	Threshold settings, embedding configuration, inference defaults, and load-test settings.
tests/	Unit tests and a small smoke or integration test for the inference path.
Dockerfile	A reproducible container build path that lets a grader run the system from a clean clone.
reports/ or artifacts/	Optional lightweight runtime summaries, logs, or notes that help the grader inspect the milestone output.

Names are suggestions, not hard requirements. What matters is that the pipeline structure is understandable, and the runnable path is obvious.

4. .gitignore expectations for Milestone 3

- Ignore downloaded dataset caches, large model assets, and temporary generated outputs unless you intentionally include a small, committed example for grading.
- Ignore local virtual environments, caches, notebook checkpoints, operating system files, and temporary Docker or profiling artifacts.

- Commit the code, configs, tests, README, Dockerfile, and any lightweight summary files that are genuinely helpful for grading.
- Do not commit large local-only outputs simply because they were produced while testing.

5. Step-by-step: Milestone 3 tasks

Step 5.1 Lock the Milestone 2 evaluation backbone

Before changing the representation, make sure your Milestone 2 evaluation logic is stable. You should already know which pair files you are using, which split is used for threshold selection, which split is used for final reporting, what score direction your system uses, and how the decision rule is applied.

- Reuse the deterministic pair files from Milestone 1 unless you have a clearly documented reason to regenerate them.
- Carry forward the Milestone 2 evaluation discipline, including split usage and the threshold-selection rule.
- Do not assume the old numerical threshold is still valid after you switch representations. Preserve the logic, not necessarily the old threshold value.

Step 5.2 Add the embedding stage

This is the most important new requirement in Milestone 3. Your pair-level inference path should now look like this: load image A and image B, preprocess both deterministically, generate embedding A and embedding B, compute a similarity score between those embeddings, apply the operating threshold, compute or derive a calibrated confidence, and return the result.

- The embedding stage must be explicit in the code structure.
- The similarity score must be computed from embeddings.
- The embedding dimensionality and any preprocessing assumptions should be documented.
- This milestone is not primarily about training a new face model from scratch. A documented embedding source is acceptable as long as the pipeline is clear, reproducible, and honestly described.

Recommended embedding-model choices

Use a documented face-embedding model in Milestone 3. The goal is to replace the earlier weak placeholder representation with a face-oriented embedding stage, not to continue using only resized pixel vectors or other simple fixed numeric representations.

Strong reasonable options include the following:

- ArcFace or InsightFace-style embeddings. These are strong modern choices if you want a high-quality identity-focused representation.
- FaceNet or InceptionResnetV1-based embeddings. These are widely used and easy to explain as a classical face-verification baseline.

- Dlib face-recognition embeddings. These are simpler to integrate and are acceptable if your emphasis is more on clean system packaging than on pushing representation quality.
- MobileFaceNet or another lightweight face-embedding model. This is a good option if you want a smaller and faster deployment-oriented pipeline.

You may also explore another face-embedding model if it is clearly documented and reproducible. If you compare multiple models, choose one as your main Milestone 3 system, explain why you chose it, and keep the evaluation and packaging path clear.

Step 5.3 Re-select the operating threshold for the embedding-based system

Once the representation changes, the score distribution usually changes as well. That means the operating threshold used by the CLI should be selected for the embedding-based system, not copied blindly from the old baseline.

- Reuse the same threshold-selection discipline from Milestone 2, such as selecting the threshold on a non-test split using a stated rule.
- Record the threshold used by the Milestone 3 CLI in a config file, notes, or README so it is easy to inspect.
- Do not tune the threshold on the final test split.

Step 5.4 Keep the inference stages separated

Milestone 4 later asks for a latency breakdown of preprocessing, embedding, and scoring. Structure your Milestone 3 code now so those stages are visible and not hidden inside one monolithic script.

- Preprocessing
- Embedding generation
- Similarity scoring
- Threshold decision
- Confidence computation
- Latency measurement

This separation will make your code easier to test, debug, package, and profile.

Step 5.5 Build the CLI inference path

The default Milestone 3 interface is a CLI. Your CLI should support at least one clean inference path, such as one pair at a time using two image paths or a small batch file containing multiple pairs.

- For each pair, print the two inputs or a pair identifier.
- Print the similarity score.
- Print the threshold used for the decision, even though this is not strictly required. It improves transparency and debugging.
- Print the binary decision.
- Print the calibrated confidence.
- Print latency for that inference.

Step 5.6 Define and document confidence

The project requires calibrated confidence, so your CLI should not output an unexplained number labeled "confidence." Your README should clearly state what the confidence value means and how it is computed.

- A confidence value can be derived from the score relative to the threshold or from another clearly documented transformation.
- The rule does not need to be perfect, but it must be reproducible and explained.
- State the output range and interpretation clearly.

Step 5.7 Package the system in Docker

Your Docker setup should let a grader build and run the system from a clean clone without hand-fixing local dependencies.

- Include a Dockerfile.
- Install the required dependencies inside the container.
- Make sure the CLI is runnable inside the container.
- Test the exact README commands before creating the Milestone 3 tag.

Step 5.8 Add a concurrency or load test

Milestone 3 requires a concurrency or load-test script that runs a feasible local workload and reports latency distribution and throughput. This does not need to be a huge benchmark. It needs to show that your inference system behaves sensibly when multiple requests are processed in parallel.

- Use a deterministic set of input pairs for the load test.
- Run a configurable number of workers, processes, or requests.
- Record per-request latency.
- Report total requests processed, total wall-clock time, throughput, and at least one tail-latency statistic such as p95.
- Include failure counts if any requests fail.

Step 5.9 Preserve reproducibility under the new inference path

Your README should allow a grader to build the Docker image or create the environment, run a sample CLI inference, run the load test, and locate the relevant outputs or summaries.

- Before creating v0.3, test the instructions from a fresh clone.
- Keep the artifact paths obvious.
- Do not rely on hidden local assumptions such as absolute file paths or manually edited environment state.

Step 5.10 Add tests and tag the release only after the runnable path passes

Milestone 2 already established the expectation of tests. Milestone 3 should keep that discipline, now focused on inference and packaging.

- Good unit-test targets include threshold application, similarity computation on toy embeddings, confidence calculation, and CLI argument validation.
- Include one small smoke or integration test that confirms the main inference path completes successfully.
- Create the Milestone 3 tag only after the environment setup or Docker build, sample CLI inference, load-test script, and required tests all work from a clean clone.

6. Recommended README content for Milestone 3

The README is part of the milestone. It should contain a short project overview and a clear How to run section with copy-pastable commands. For Milestone 3, it also needs to explain the inference interface and Docker workflow.

- Project overview: a short paragraph describing what the system does and what Milestone 3 adds.
- Pipeline summary: preprocessing, embedding generation, similarity scoring, thresholding, confidence computation.
- How to run: Docker build commands, sample CLI inference, optional batch inference, load-test commands, and tests.
- Artifact locations: where to find runtime summaries, logs, or any small committed examples.
- Design notes: embedding source or model choice, threshold used by the CLI, and how confidence is computed.

7. Common mistakes to avoid

- Packaging the old weak baseline without adding an embedding stage.
- Hiding preprocessing, embedding, scoring, and thresholding inside one large script with no clear structure.
- Printing a confidence number without explaining what it means.
- Reporting only an average latency and forgetting throughput or tail-latency behavior.
- Using a load test that is too large or unrealistic for local grading.
- Writing README commands that were never actually tested from a fresh clone.
- Changing the project flow substantially without documenting the design clearly.
- Building a Docker image that works only with manual local fixes.

8. One acceptable Milestone 3 implementation plan

There is more than one valid way to complete Milestone 3. The phases below describe one clean and reasonable path.

Phase	Main goal
Phase A	Carry forward the verified backbone from Milestone 2, including deterministic pairs and threshold-selection discipline.
Phase B	Add preprocessing and embedding extraction, then compute similarity from embeddings.
Phase C	Expose pair-level inference through a clean CLI output path.
Phase D	Package the runnable system in Docker and test the exact README commands from a fresh clone.
Phase E	Run the load test, summarize throughput and latency, and save a lightweight result summary.
Phase F	Run tests, verify reproducibility, and tag the exact runnable commit as v0.3.

9. Suggested Milestone 3 artifacts

- Dockerfile
- CLI script or wrapper
- Load-test script
- Config file for threshold and inference settings
- Small runtime summary file, such as CSV or JSON
- Unit tests
- One small inference smoke test
- README with Docker and CLI commands

Optional but useful artifacts include a short runtime summary note in a reports folder and a small, committed example of CLI output.

10. Final summary

Milestone 3 is where your face verification project becomes a deployable inference system. Carry forward the deterministic pairs and thresholding discipline from Milestones 1 and 2, replace the earlier placeholder representation with an embedding-based one, expose a clean CLI inference interface, package it in Docker, and measure how it behaves under concurrent use.

By the end of this milestone, a grader should be able to build your system from a clean clone, run pair-level inference, inspect decision, score, confidence, and latency, and reproduce your runtime behavior using the README commands.